

Adaptive Admission for Small Object Caches

A fictional paper written for the PageWise evaluation corpus. No results in it were measured.

Abstract. Object caches in front of storage services are dominated by small, short-lived objects whose reuse distance is hard to predict. We describe *Sieve-A*, an admission filter that decides whether a newly fetched object should enter the cache at all, and we evaluate it against LRU, ARC and a TinyLFU-style frequency sketch on four production-like traces. Sieve-A lowers the byte miss ratio by 7.4% on average and by up to 18.9% on the most skewed trace, while using a fixed 16 KiB of metadata per million cached objects. We also report two negative results: admission control offers little on traces whose working set already fits in memory, and the filter’s benefit disappears when object sizes vary by more than three orders of magnitude.

1 Introduction

A cache that admits every object it fetches is making a bet: that the object will be requested again before it is evicted. For large, popular objects the bet usually pays off. For the long tail of small objects that most real traces contain, it frequently does not—and each lost bet costs an eviction of something that might have been reused. The classic response has been to improve the eviction policy. This paper takes the other route and asks whether an object should be admitted in the first place.

Admission filters are not new. TinyLFU keeps an approximate frequency count for every key and admits a newcomer only if it is more popular than the victim it would displace. The difficulty is that approximate counters must be aged, and the aging schedule interacts with the trace in ways that are difficult to tune offline. Our contribution is a filter whose only tunable parameter, the sampling rate ρ , can be adjusted online from the observed hit ratio.

We make three claims. First, on traces with a Zipf exponent above 0.9, admission control matters more than the choice of eviction policy. Second, a filter with a fixed memory budget can match an ideally tuned TinyLFU without offline tuning. Third, the gains are confined to workloads where most requests fall on objects smaller than 4 KiB; we find no benefit, and occasionally a small loss, elsewhere.

2 Background

Let a trace be a sequence of requests $r_1, \dots, r_n$, each naming an object with a size in bytes. A

cache of capacity C bytes serves a request from memory if the object is resident and fetches it from the backing store otherwise. We report the *byte miss ratio*, the fraction of requested bytes that had to be fetched, because it is the quantity that determines backend load and egress cost.

LRU evicts the least recently used object. ARC balances recency and frequency with two lists whose relative sizes adapt to the trace. TinyLFU adds a frequency sketch in front of an eviction policy and rejects newcomers whose estimated frequency is lower than the victim’s. All three are well understood; we use the reference implementations described in the appendix without modification.

A useful way to think about admission is in terms of *reuse distance*: the number of distinct bytes requested between two accesses to the same object. An object whose reuse distance exceeds C will be evicted before it is reused under any policy that is not clairvoyant, so admitting it can only do harm. The filter’s task is to estimate, from a small sample, which newcomers are likely to fall into that category.

3 Design of Sieve-A

Sieve-A samples one request in every $1/\rho$ and records, for each sampled key, the logical time at which it was seen. When the same key is sampled again, the filter computes an estimated reuse distance and adds it to a small histogram with 64 logarithmic buckets. A newcomer is admitted if its size class has a median estimated reuse distance below the current cache capacity.

$$\text{admit}(o) \Leftarrow \text{median}_{k(o)}(\hat{d}) < C \cdot (1 - \varepsilon)$$

Here $k(o)$ is the size class of object o and ε is a safety margin, fixed at 0.1 in all experiments. The histogram is aged by halving every bucket after each 2^{20} sampled requests; unlike TinyLFU’s counter reset, halving is applied to a structure that is already an estimate of a distribution, so it does not erase the ordering between size classes.

The sampling rate starts at $\rho = 1/64$. Every epoch the filter compares the observed hit ratio with the hit ratio of the previous epoch; if it has fallen by more than one percentage point, ρ is doubled (up to $1/8$) so that the histogram reacts faster, and otherwise it is halved (down to $1/512$) to save memory bandwidth. This is the only feedback loop in the design, and it converged within ten epochs on every trace we tried.

Memory use is fixed. Each size class holds a 64-bucket histogram of 32-bit counters, and there are 32 size classes, so the histograms occupy 8 KiB; the sample table holds 1,024 entries of 8 bytes each. Together this is the 16 KiB quoted in the abstract, independent of the number of cached objects.

4 Methodology

We use four traces. **WEB** is a synthetic web-proxy trace with a Zipf exponent of 0.8 and log-normally distributed sizes. **KV** models a key-value store serving session tokens: 92% of objects are under 512 bytes. **CDN** is dominated by image thumbnails with a Zipf exponent of 1.1. **MIX** interleaves the other three at equal request rates, producing sizes that range from 40 bytes to 60 MiB.

Each trace contains 200 million requests. Cache capacity is set to 1%, 2% and 5% of the trace’s unique bytes. Every configuration is run five times with different random seeds for the sampler, and we report the mean; the standard deviation never exceeded 0.3 percentage points and is omitted from the tables.

Trace	LRU	ARC	TinyLFU	Sieve-A
WEB	41.2	38.9	36.1	35.4
KV	27.6	25.3	21.8	19.7
CDN	33.0	31.4	26.2	22.4
MIX	45.8	44.1	43.9	44.6

Table 1: Byte miss ratio (%) at a capacity of 2% of unique bytes. Lower is better.

5 Results

Table 1 summarises the main result. On KV and CDN, Sieve-A outperforms TinyLFU by 2.1 and 3.8 percentage points respectively; relative to LRU, the reduction on CDN is 32%. On WEB the difference between the two admission filters is within 0.7 points. On MIX, Sieve-A is slightly worse than TinyLFU and ARC, which we examine in Section 6.

The averaged improvement of 7.4% quoted in the abstract is the mean relative reduction of byte miss ratio against the best non-admission policy (ARC), taken over all four traces and all three capacities. The largest single improvement, 18.9%, occurs on CDN at a capacity of 1%.

Throughput is not affected in any way we could measure. The filter adds one hash computation and, for sampled requests, one table lookup; at $1/64$ sampling this amounts to under 40 ns per request on the test machine, compared with a median fetch latency of 1.8 ms from the backing store.

6 Where admission does not help

Two situations defeat the filter. The first is unsurprising: when the working set fits in the cache, every object is eventually reused and rejecting any of them is a loss. At a capacity of 5% on WEB, all four policies are within 1.5 points of one another.

The second is more instructive. On MIX, a single size class may contain both a 40-byte session token and a 3 KiB thumbnail, because the classes are logarithmic in size and the traces overlap. The median reuse distance of such a class describes neither population, and the filter rejects tokens that would have been reused. We tried doubling the number of size classes; it

recovered about half of the loss and doubled the memory footprint, which we do not consider a good trade.

Footnote: an earlier version of this paper reported a 9.1% average improvement. That figure included a WEB run with a misconfigured capacity and has been withdrawn.

7 Related work

Size-aware admission has a long history in web caching, where policies such as GDSF weigh an object’s size against its frequency. Machine-learned admission filters have been proposed that predict reuse from request features; they report larger gains than ours on the traces they were trained on, but require offline train-

ing and a model that must itself be kept current. Sieve-A is deliberately simpler: it has one feedback loop and a fixed memory cost, and it can be disabled at run time without flushing the cache.

8 Conclusion

Admission control is worth more than eviction policy on skewed traces of small objects, and it can be done with a fixed, small amount of memory and a single online parameter. It is worth little when the working set fits in memory, and it can hurt when a size class mixes unrelated populations. We recommend enabling it per tenant, where the size distribution is known to be narrow, rather than globally.